

GPU 算子开发

手撕不止能写：先定 partition，再谈搬运、同步、规约和性能

GEMV / GEMM

Nsys → NCU

FlashAttention

Fusion

岗位能力画像

岗位画像：重点看线程映射、memory hierarchy、profiling 判断与融合取舍，而不是背 API。

高频深挖：题目背后的判断

Nsys 和 NCU 怎么分工？

Nsys 负责时间线、CPU/GPU、通信和 bubble；NCU 负责热点 kernel 的资源与 stall。

先定位层级，再选择 profiler。

GEMM 为什么要 tiling？

把 HBM→shared→register→matrix unit 的复用画出来，才能解释数据复用与 coalescing。

tile 是数据复用和资源预算的共同选择。

bank conflict 怎么答？

说明地址映射到 bank 的方式，再给 padding、swizzle、重排及其代价。

不是任何 shared memory 都需要 padding。

fusion 为什么不总是快？

要同时算少掉的 launch/HBM 往返与新增的 register/smем 压力、occupancy 和依赖。

融合是资源交换，不是免费优化。

项目叙事的通用深挖链

问题与约束

证据与机制

方案与取舍

验证与边界

答题提醒

不要从‘用了什么库/指令’开始；先固定工作负载、指标和约束，再说明数据流/资源/依赖，最后给出正确性、性能和反例边界。

把一句结论变成可复验的事实

现象

固定 workload / 阶段 / 指标，明确‘慢’或‘错’在哪里。

证据

用 trace、counter、对照或最小复现排除相邻假设。

机制

说明资源、数据流或依赖如何导致这个现象。

验收

数值、边界、重复性和 E2E 共同成立后再下结论。

一题答透：Profiling 漏斗：从时间线到一个可行动的假设

问题 “一个 kernel 很慢，如何避免盲目改 tile？”

1. E2E / Nsys

确认热点的真实占比，分 launch-bound、长 kernel、同步/通信等待和负载不均。

2. Roofline / NCU

看 Tensor pipe、DRAM/L2、SM active、eligible warps、register/smem 与 stall reason。

3. 写假设

每个假设只对应一种可删除的成本：无效 tile、HBM 往返、bank conflict、等待或 launch。

4. 回归

shape×dtype×batch 取中位/分位，配数值 diff、race/mem check 与 E2E。

把推理链画出来



常见误区

× 只报 occupancy，不看实际 active warps 和 stall。

× 拿单一 microbenchmark 代表模型收益。

× 先写 shared memory trick，后想线程映射。

术语速懂 | 面试怎么答

术语速懂

Roofline

用算术强度与硬件计算/带宽上限判断更接近算力墙还是带宽墙。

Online Softmax

分块维护 running max 与 normalizer，避免完整 score/probability 矩阵落到 HBM。

面试中怎么答

① 先估算工作量和访存量，并用 profiler 判断受限类型。

② 再从线程映射、数据复用、tile 和同步解释改动为什么可能有效。

③ 说明 tail、NaN、数值 diff、多 shape 和 E2E 的验收口径。

白板 / 手撕：白板题：GEMV 与 Row-Wise Max

题面 请为 $A[N,K] \times x[K] \rightarrow y[N]$ 设计线程映射，并解释向量化 load、tail 和极端 shape。

CUDA C++ · warp-per-row GEMV

核心实现

```
01 __global__ void gemv_warp_rows(const float* A, const float* x,
02     float* y, int N, int K) {
03     int lane = threadIdx.x & 31;
04     int row = blockIdx.x * 8 + (threadIdx.x >> 5);
05     if (row >= N) return;
06     float sum = 0.f;
07     for (int k = lane; k < K; k += 32)
08         sum += A[row * K + k] * x[k];
09     for (int d = 16; d; d >>= 1)
10         sum += __shfl_down_sync(0xffffffff, sum, d);
11     if (lane == 0) y[row] = sum;
12 }
13 // launch: <<<ceil_div(N, 8), 256>>>
```

写代码时怎么讲

1. 先写 baseline：一个 block/row，grid-stride 读取 K，寄存器累加。
2. warp shuffle 做规约，多 warp 时 shared 存 warp partial。
3. N 大/K 小：warp-per-row；N 小/K 大：split-K 或多阶段规约，并说明数值限制。
4. vectorized load 需 alignment；K 非整除必须 mask，避免越界。

实现重点

- 一条 warp 对应一行输出，天然覆盖 K 尾部。
- 连续布局、对齐且 K 合适时才加 vectorized load 快路径。
- N 很小/K 很大时比较 split-K 与二次规约的成本。

边界、测试与追问

最少要说明的验证

- 一条 warp 对应一行输出，天然覆盖 K 尾部。
- 连续布局、对齐且 K 合适时才加 vectorized load 快路径。
- N 很小/K 很大时比较 split-K 与二次规约的成本。

面试继续追问

- softmax 如何保证数值稳定？
- FlashAttention 如何避免完整 score 矩阵落 HBM？
- warp specialization 在什么条件下有收益？